

PDF 3 of 3 — Interview & Practical Notes

Scalability

Interview-focused learning material built on top of the source lesson: mental models, real-world examples, trade-offs, misconceptions, interview questions and model answers, follow-up drills, practical exercises, and a senior-engineer perspective.

Beginner → Senior

India · UAE · Saudi

SDE-2 / SDE-3 / Tech Lead

Contents

1. Topic Overview
2. Simple Mental Model
3. Key Points to Remember
4. Real-World Examples
5. When to Use / When NOT to Use
6. Trade-offs
7. Common Misconceptions
8. Interview Questions (India · UAE · Saudi)
9. Interview Answers
10. Follow-up Questions
11. Practical Design Exercises
12. Quick Interview Revision Sheet
13. Senior Engineer Perspective

1. Topic Overview

Scalability is a system's ability to keep working well as demand grows — more users, more requests per second, more data — by adding resources rather than by rewriting everything from scratch. It exists because every system starts on finite hardware, and traffic almost never stays where it started. A service built for 100 users a day eventually meets 100,000 users a day, and the question is whether it bends or breaks.

The problem scalability solves is capacity under growth: how do you serve more without a proportional collapse in latency, or without the system falling over completely. There are exactly two levers, and the source lesson names both: make the existing machine bigger (vertical scaling / scaling up), or add more machines and spread the work across them (horizontal scaling / scaling out).

Scalability matters in System Design because it is one of the first things an interviewer probes — it forces you to reveal whether you understand where state lives, where the bottleneck will move to next, and what breaks first under load. It is also foundational: almost every other "-ility" in distributed systems (availability,

reliability, fault tolerance) is built on top of a scaling decision. You cannot design for high availability, for instance, without first deciding how the system scales, because availability in a modern system usually comes from having many redundant, horizontally-scaled instances.

In a larger architecture, scalability decisions ripple outward. Choosing horizontal scaling for your application tier immediately creates new requirements: a load balancer to distribute requests, a way to avoid sticky per-instance state, and eventually a data tier that can also scale (through caching, read replicas, or sharding). Choosing vertical scaling defers those requirements — which is sometimes exactly the right call for a small, early-stage system, and sometimes a trap for a system that will need to scale further later.

Beginner framing

Scalability = handling more traffic. Two options: bigger machine, or more machines.

Senior framing

Scalability = a deliberate trade-off between hardware simplicity (vertical) and distributed-systems complexity (horizontal), chosen based on growth rate, availability needs, and how much of the system can be made stateless or partitioned.

2. Simple Mental Model

The analogy: checkout counters at a supermarket

Imagine a supermarket with one checkout counter. When the store gets busy, you have two choices. You can train that one cashier to scan items faster and give them a bigger, faster till — that is **vertical scaling**. Or you can open more checkout counters and let customers queue at whichever one is free — that is **horizontal scaling**.

The one-cashier approach is simple: there's only one queue, one till, nothing to coordinate. But there's a limit to how fast one person can scan items, and if that cashier goes on a break, the whole store stops selling anything. The many-counters approach has no real ceiling — you can keep opening counters — but now you need someone directing customers to a free counter (a load balancer), and if a customer's loyalty card or shopping cart is tied to "counter 3" specifically, the system breaks the moment counter 3 is busy or down.

Beginner understanding

Vertical = one bigger/faster machine.
Horizontal = many machines sharing the load.
Horizontal is "the scalable one"; vertical "runs out of room."

Interview-level understanding

Horizontal scaling only pays off if the "customer" (request) doesn't care which "counter" (node) serves it. That requires statelessness at the app tier and a partitioning/replication strategy at the data tier. Without those, adding machines just adds idle machines — the real bottleneck (a single database, a shared lock, a sticky session) is untouched. This is the gap between "I added servers" and "I actually scaled the system."

3. Key Points to Remember ☆

- ★ **Definition:** Scalability is handling increasing workload by adding resources — either bigger resources (vertical) or more resources (horizontal).
- ★ **Horizontal scaling (scaling out):** add more nodes; workload is distributed across them; the classic approach for distributed systems (e.g. Cassandra, MongoDB).
- ★ **Vertical scaling (scaling up):** increase one node's CPU/memory/storage; simple but bounded by that machine's ceiling (e.g. resizing a MySQL instance).
- ★ **Statelessness is the enabler:** horizontal scaling only works cleanly when any node can serve any request — which means session/user state must live outside the instance (shared cache, database, token).
- ★ **Vertical scaling usually needs downtime** to resize, and it concentrates everything on one machine — a single point of failure (SPOF).
- ★ **Horizontal scaling needs a load balancer** (or equivalent routing/service-discovery layer) to actually spread requests across the pool.
- ★ **Diminishing returns exist on both sides:** vertical scaling gets disproportionately expensive near the top of the instance-size curve; horizontal scaling gets diminishing throughput gains once a shared resource (a single DB, a lock, cross-node chatter) becomes the new bottleneck — this is the practical shape of Amdahl's Law / the Universal Scalability Law.

- ★ **Scaling compute is easier than scaling data.** Stateless app servers scale out almost for free; databases need sharding, partitioning, or replication, which is where most of the real engineering effort goes.
- ★ **Commonly forgotten:** adding more application servers in front of a single, un-partitioned database does not make the system horizontally scalable — it just moves the bottleneck to the database and adds connection-pool pressure.
- ★ **Interview point:** "I would scale horizontally" is an incomplete answer. State where the state lives, and what happens on node failure, before claiming the system scales.

4. Real-World Examples

These are well-known, publicly documented patterns used to illustrate the concept — not confirmed internal architecture of any specific company unless stated as such.

Databases and caches

Cassandra and **MongoDB** (the examples the source lesson itself gives) are built around horizontal scaling: data is partitioned across nodes using a partition/shard key, and adding nodes redistributes data automatically (consistent hashing in Cassandra, chunk-based sharding in MongoDB). **MySQL** and **PostgreSQL** are traditionally scaled vertically first (bigger instance, more RAM for buffer pool/cache) because horizontal write-scaling for a relational database requires extra tooling — read replicas for read scaling, and application-level or proxy-level sharding (e.g. Vitess for MySQL, Citus for PostgreSQL) for write scaling. **Redis** is commonly run as a single powerful instance (scaled vertically, since a single Redis process is largely single-threaded for command execution) until Redis Cluster is introduced to shard keys horizontally across nodes.

Cloud infrastructure

AWS EC2 Auto Scaling Groups behind an Application Load Balancer are the textbook implementation of dynamic horizontal scaling — instances are added or removed based on load, matching the source lesson's point that "capacity can be added while the system runs." Resizing an **AWS RDS** or **Azure SQL** instance class to get more vCPU/RAM is the textbook implementation of vertical scaling, and it typically requires a brief failover or restart — matching the lesson's point that vertical scaling "usually involves downtime." **GCP** and **Azure** offer equivalent managed autoscaling groups and VM resizing.

Containers and orchestration

Kubernetes exposes this exact choice as two separate controllers: the Horizontal Pod Autoscaler (HPA) adds more pod replicas under load, while the Vertical Pod Autoscaler (VPA) increases the CPU/memory request of existing pods. Seeing both exist as first-class, separate features is a good confirmation that this is a genuine architectural fork, not just terminology.

Products and companies

Netflix and **Amazon** are widely known, from public engineering blogs and conference talks, for running large fleets of horizontally-scaled, stateless microservices behind load balancers — an architecture that assumes individual instances will fail routinely, which is precisely why statelessness and horizontal redundancy matter together. Consumer platforms with sharp traffic spikes — ride-hailing apps like **Uber** or **Careem**, food delivery apps like **Talabat**, **Zomato**, or **HungerStation**, and e-commerce platforms during sale events (**Flipkart**'s Big Billion Days, **Noon**'s sale days) — are commonly used in interviews as scenario prompts precisely because they need to add capacity horizontally and quickly during predictable demand spikes. Payments platforms such as **Razorpay**, **PhonePe**, or **PayTM** are good discussion examples for the "vertical-first, then horizontal" journey on the database side, since financial consistency requirements make sharding a transactional database a serious, carefully-staged undertaking rather than a default choice.

Caution

Treat the company examples above as illustrations of well-known industry patterns, not as verified statements about any company's current, non-public internal architecture.

5. When to Use / When NOT to Use

When Horizontal Scaling Makes Sense

- Traffic is variable, growing, or hard to predict, and you want to add/remove capacity dynamically.
- High availability matters — you cannot afford a single machine to be a single point of failure.
- The application tier is stateless, or can reasonably be made stateless.
- You want to use commodity hardware / smaller cloud instances instead of paying a premium for the largest available machine.
- You need zero-downtime deployments (roll instances in and out of the pool one at a time).

When Horizontal Scaling Is Overkill

- Traffic is small, stable, and well within a single machine's capacity — added complexity buys nothing.
- The application is a tightly-coupled legacy monolith where making it stateless is a large, risky rewrite for a problem you don't have yet.
- Team size and operational maturity can't yet support the extra moving parts (load balancer, service discovery, distributed monitoring).
- Licensing is per-core/per-instance and horizontal scaling would multiply license cost faster than it multiplies capacity.
- The workload has a bottleneck that doesn't parallelize (e.g. a strictly sequential batch job) — more nodes won't help that specific step.

When Vertical Scaling Makes Sense

- Early-stage systems, low/predictable traffic, or internal tools where simplicity beats theoretical ceiling.
- A quick, low-risk way to buy time before a bigger architectural change.
- Systems that are inherently hard to distribute (e.g. some single-threaded workloads, certain in-memory data stores).
- Reducing operational overhead is more valuable than reducing the theoretical scaling ceiling.

When Vertical Scaling Is Risky

- Growth shows no sign of slowing and will likely exceed the largest available machine within a foreseeable horizon.
- The system needs high availability — one machine is always a single point of failure, no matter how big.
- Frequent resizing means frequent downtime, which the business cannot tolerate.

It depends on the requirements

There is no universally "correct" choice between horizontal and vertical scaling. The right answer depends on current scale, growth trajectory, availability requirements, team maturity, and cost constraints — which is exactly what a senior candidate is expected to probe for before committing to an architecture.

6. Trade-offs

Option	Advantages	Disadvantages	Best Use Case
Horizontal Scaling	Near-limitless capacity; no single point of failure; enables rolling, zero-downtime deployments; can use cheaper commodity instances	Requires statelessness or data partitioning; introduces distributed-systems complexity (coordination, consistency, network partitions); higher operational overhead (orchestration, service discovery, monitoring many nodes)	High-traffic, elastic, high-availability systems: web/mobile backends, streaming platforms, e-commerce during sales
Vertical Scaling	Simple — no code or architecture changes; easy to reason about and debug; lower day-to-day operational overhead	Hard ceiling at the biggest available machine; single point of failure; resizing usually needs downtime; cost grows non-linearly at the high end	Small-to-medium systems, legacy monoliths, databases not yet ready to shard, short-term fixes

Dimension by dimension

Performance: vertical scaling can improve single-request latency (faster CPU, more cache-friendly memory) with zero network overhead; horizontal scaling improves aggregate throughput but each request may now cross a network hop (to a load balancer, to a shared cache, to a partitioned database) that a single machine wouldn't have.

Scalability: horizontal wins for headroom; vertical wins for how quickly you can apply it without any redesign.

Availability & Reliability: horizontal scaling, done right, removes the single point of failure that vertical scaling always carries — but only if the horizontal fleet also has redundancy in the data tier, not just the app tier.

Consistency: vertical scaling keeps a single copy of data, so consistency is "free." Horizontal scaling of stateful systems (databases, caches) forces an explicit choice — usually framed by the CAP theorem — between strict consistency and availability during network partitions.

Complexity, Operational Overhead & Development Effort: all three are lower for vertical scaling and higher for horizontal scaling. This is the real price of horizontal scaling and the reason it isn't the default choice for every system regardless of size.

Cost: vertical scaling has a steep, non-linear cost curve at the top of the instance range; horizontal scaling can be cheaper per unit of capacity (many small instances) but adds indirect costs — networking, orchestration tooling, and engineering time.

Maintainability: a vertically-scaled single instance is simpler to patch and monitor; a horizontally-scaled fleet needs automation (infrastructure as code, centralized logging/metrics) to stay maintainable at all — manual maintenance across dozens of nodes doesn't scale even if the traffic does.

7. Common Misconceptions

"Horizontal scaling always gives linear performance gains — 2x machines means 2x throughput."

WHY IT'S WRONG/INCOMPLETE

Any portion of the workload that can't be parallelized, plus coordination overhead between nodes (network calls, shared locks, a single shared database), caps the real speedup. This is Amdahl's Law in practice, and at very high node counts the Universal Scalability Law shows throughput can even fall as coordination cost overtakes the benefit of more nodes.

CORRECT UNDERSTANDING

Horizontal scaling gives diminishing returns unless shared bottlenecks are actively removed — through partitioning data, avoiding cross-node locking, and keeping inter-node communication minimal.

INTERVIEW-LEVEL NUANCE

Be ready to name the bottleneck that will appear next (usually the database or a shared cache) rather than assuming "add more servers" is a complete answer on its own.

"MongoDB and Cassandra always scale horizontally, with no trade-offs."

WHY IT'S WRONG/INCOMPLETE

Both support horizontal scaling as a capability, not a guarantee. It depends heavily on choosing a good shard/partition key. A poor key choice creates hot partitions — one node doing disproportionate work while others sit idle — and cross-shard queries or joins become expensive or need to be handled at the application level.

CORRECT UNDERSTANDING

Horizontal scalability in these databases has to be designed for; it doesn't happen automatically just by choosing the technology.

INTERVIEW-LEVEL NUANCE

Mention that Cassandra and MongoDB also make explicit CAP-theorem trade-offs (e.g. Cassandra's tunable consistency) — horizontal scaling doesn't remove the consistency-availability trade-off, it just moves it into your data-modeling decisions.

"Vertical scaling is outdated and nobody uses it in modern systems."

WHY IT'S WRONG/INCOMPLETE

Many production systems still rely on vertical scaling for their primary relational database, an in-memory cache, or a single-threaded service, because horizontally scaling stateful systems is genuinely hard engineering work.

CORRECT UNDERSTANDING

Horizontal and vertical scaling are complementary, not competing philosophies. A common real-world path is: scale vertically first because it's fast and free of redesign, and move to horizontal scaling once the vertical ceiling, cost curve, or availability requirement forces the issue.

INTERVIEW-LEVEL NUANCE

Saying "I'd scale vertically first because it needs no architecture change, then re-evaluate" often reads as more senior than jumping straight to a fully distributed design for a system that doesn't need it yet.

"Stateless just means the service doesn't use a database."

WHY IT'S WRONG/INCOMPLETE

Statelessness is about where per-request/session context lives, not about whether a database is involved at all.

CORRECT UNDERSTANDING

A service is stateless when any instance can handle any request because the request carries (or can fetch) everything it needs — from a database, a token, or a shared cache — rather than relying on data stored only on that one instance's memory or disk.

INTERVIEW-LEVEL NUANCE

Be precise: "stateless app tier, stateful data tier" is usually the accurate description of a well-designed horizontally-scaled system, not "no state anywhere."

"Adding a load balancer in front of my servers makes the system horizontally scalable."

WHY IT'S WRONG/INCOMPLETE

A load balancer only distributes traffic across whatever is behind it. If sessions are pinned to specific instances, or every instance still talks to one un-partitioned database, the real bottleneck hasn't moved.

CORRECT UNDERSTANDING

True horizontal scalability requires the whole request path — app tier, cache, and data tier — to support scale-out, not just the entry point.

INTERVIEW-LEVEL NUANCE

A strong answer traces the request path end-to-end and names where the design still has a single point of contention, rather than stopping at "we'd add a load balancer."

8. Interview Questions

Questions below are organized by difficulty level. Region-relevant company names are noted where the scenario is a natural fit for that market's product companies (e-commerce, ride-hailing, food delivery, fintech) — this reflects the kind of system those companies operate publicly, not a claim that a specific company asked this exact question.

L1 Beginner

COMMON INTERVIEW QUESTION

What is scalability in system design, and why does it matter?

COMMON INTERVIEW QUESTION

What is the difference between horizontal scaling and vertical scaling?

COMMON INTERVIEW QUESTION

Give one example each of a system that typically scales horizontally and one that typically scales vertically.

FREQUENTLY REPORTED QUESTION

What is a load balancer, and why does horizontal scaling need one?

L2 Intermediate

FREQUENTLY REPORTED QUESTION · relevant across India/UAE product companies (Amazon, Flipkart,

PhonePe, Careem, Noon)

Why is statelessness important for horizontal scaling, and how do you handle user sessions in a horizontally scaled web application?

INTERVIEW-STYLE QUESTION

What challenges would you face trying to horizontally scale a relational database like MySQL, compared to a database like Cassandra or MongoDB?

COMMON INTERVIEW QUESTION

When would you choose vertical scaling over horizontal scaling for a real project?

INTERVIEW-STYLE QUESTION

How does Kubernetes' Horizontal Pod Autoscaler differ from its Vertical Pod Autoscaler, and when would you use each?

L3 Senior

SENIOR-LEVEL FOLLOW-UP

Why doesn't doubling the number of servers always double throughput? How would you explain the diminishing returns of horizontal scaling?

SENIOR-LEVEL FOLLOW-UP · relevant to fintech (Razorpay, PhonePe, PayTM, Tabby, Tamara)

How would you design a sharding strategy for a horizontally scaled transactional database, and what goes wrong with a poorly chosen shard key?

SENIOR-LEVEL FOLLOW-UP

How do you decide what stays stateless in the application tier versus what must be stateful in the data tier?

SENIOR-LEVEL FOLLOW-UP

What operational complexity does horizontal scaling introduce — service discovery, deployment, observability — and how would you manage it at scale?

L4 Scenario-Based

SCENARIO-BASED QUESTION

Traffic to your API suddenly increases 10x — for example, a flash sale on an e-commerce platform like Flipkart or Noon. What would you change, and in what order?

SCENARIO-BASED QUESTION

One node in your horizontally scaled fleet fails in the middle of the night. Walk through exactly what happens, end to end.

SCENARIO-BASED QUESTION

Your database has become the bottleneck even though your application servers have plenty of headroom. What would you do?

SCENARIO-BASED QUESTION

How would you make a currently single-machine, vertically-scaled service highly available without a full rewrite?

SCENARIO-BASED QUESTION · relevant to Talabat, Zomato, Swiggy, HungerStation, Jahez order-spike scenarios

A food-delivery order service sees predictable but sharp spikes at lunch and dinner. How would you reduce latency and avoid over-provisioning for the other 20 hours of the day?

9. Interview Answers ☆

Concise, senior-level answers for the highest-value questions above. Each demonstrates the same reasoning pattern: requirement, decision, reason, trade-off, alternative.

Q: What's the real difference between horizontal and vertical scaling, and how do you decide between them?

MODEL ANSWER

Horizontal scaling adds machines and spreads the workload; vertical scaling makes one machine bigger. I'd default to vertical scaling early, because it needs no architecture change — but I'd switch to horizontal scaling once I need high availability, once traffic growth is likely to exceed the largest available instance, or once I need to deploy without downtime. The trade-off is that horizontal scaling only pays off if the app tier is stateless and the data tier can be replicated or partitioned — otherwise I've added operational complexity without actually removing the bottleneck.

Q: Why is statelessness important, and how do you handle sessions?

MODEL ANSWER

Horizontal scaling assumes any node can handle any request. If session data lives only in one instance's memory, requests from that user must always return to that instance, which defeats load balancing and creates a hidden single point of failure. I'd move session state to a shared store — a cache like Redis, or a signed token (JWT) the client carries — so the app tier stays stateless and any instance can serve any request.

Q: Traffic suddenly increases 10x. What would you change?

MODEL ANSWER

First I'd confirm the app tier is stateless and put it behind auto-scaling so it can add instances immediately — that's the fast, low-risk lever. Next I'd look at the database, since that's usually the next bottleneck: add a cache in front of hot read paths, and add read replicas if reads dominate. If writes are the problem, that's a bigger conversation about sharding, and I'd only reach for it once caching and replicas aren't enough, because sharding is the most expensive lever in terms of complexity.

Requirement: sudden, very high read traffic

Decision: introduce caching in front of the database

Reason: reduces database load and latency fastest, with the least code change

Trade-off: cache invalidation and potentially stale data

Alternative: add read replicas instead of, or alongside, a cache

Q: The database has become the bottleneck. What would you do?

MODEL ANSWER

I'd first find out whether it's a read problem or a write problem, because the fixes are different. For reads, caching and read replicas usually solve it without touching the write path. For writes, I'd look at vertically scaling the database instance first, since it's the fastest lever with no application change; if that ceiling is already close, I'd consider partitioning/sharding the data by a key that matches the access pattern — accepting that this adds real complexity around cross-shard queries and rebalancing.

Q: How would you design a sharding strategy, and what goes wrong with a bad shard key?

MODEL ANSWER

I'd pick a shard key that matches the dominant access pattern and spreads load evenly — for example, user ID or region for a consumer app. A bad shard key (like sharding by signup date, or a key with very uneven value distribution) creates hot partitions, where one shard absorbs most of the traffic while others sit idle, which defeats the purpose of sharding. I'd also plan for resharding up front, because the initial key rarely survives the system's growth unchanged.

Q: One node fails. What happens?

MODEL ANSWER

It depends entirely on the architecture I chose earlier. In a horizontally-scaled, stateless fleet behind a load balancer with health checks, that node is marked unhealthy, traffic is rerouted to the remaining nodes, and — assuming there's headroom or auto-scaling — a replacement instance is brought up automatically; users may see nothing at all. In a vertically-scaled, single-machine setup, that node failing means downtime until it's restored, which is exactly the single-point-of-failure risk vertical scaling carries.

10. Follow-up Questions 🔥

Real interviews rarely stop at the first answer. These chains train you for the drill-down.

Chain 1 — Choosing horizontal scaling

INTERVIEWER

"Why did you choose horizontal scaling?"

INTERVIEWER

"Why can't you use vertical scaling?"

INTERVIEWER

"What happens if one instance fails?"

INTERVIEWER

"How do you manage session state?"

INTERVIEWER

"What becomes the bottleneck after adding more servers?"

INTERVIEWER

"How would you scale the database?"

Chain 2 — Sharding the database

INTERVIEWER

"Why did you choose to shard the database instead of just adding read replicas?"

INTERVIEWER

"How did you pick the shard key?"

INTERVIEWER

"What happens when one shard becomes a hotspot?"

INTERVIEWER

"How do you handle a query that needs data from two different shards?"

INTERVIEWER

"How do you rebalance data when you add a new shard?"

Chain 3 — Vertical scaling as the first move

INTERVIEWER

"You said you'd scale vertically first — why not go straight to horizontal?"

INTERVIEWER

"What signal would tell you it's time to switch to horizontal scaling?"

INTERVIEWER

"What happens to your users during the resize/downtime window?"

INTERVIEWER

"How would you reduce that downtime without redesigning the whole system yet?"

11. Practical Design Exercise

Exercise 1 — Scalable Link-Shortening API

Design a link-shortening service (like a simplified Bitly) that starts with modest traffic but needs to support rapid, unpredictable growth in read traffic (link redirects far outnumber link creations).

Identify: requirements, expected traffic (reads vs writes), storage needs, architecture, likely bottlenecks, scaling strategy, failure scenarios, and trade-offs — before reading further.

Expected Thinking Process

- Clarify read:write ratio — redirects (reads) will vastly outnumber shortenings (writes), so the design should optimize reads first.
- Application tier: keep it stateless so it can scale horizontally behind a load balancer from day one — this part is cheap to get right early.
- Data tier: identify that the mapping (short code → long URL) is simple key-value data — a strong candidate for a cache in front of a database.
- Think about the write path separately: ID/short-code generation needs to avoid collisions across multiple app instances (e.g. a counter service, or a pre-generated key pool) rather than assuming a single machine generates all codes.
- Consider failure: what happens if the cache is cold or down, if one app instance dies, or if the database is temporarily unreachable.

Reference Solution

Stateless application servers behind a load balancer, auto-scaled on request volume — this handles the redirect traffic horizontally with no code-level obstacle, since redirects don't need session state. A cache (e.g. Redis) sits in front of the database for the hot path (short code → long URL lookups), since this is a read-heavy workload and most requests hit a small set of popular links. The database itself can start as a single, vertically-scaled instance with a replica for read scaling and failover; if write volume or data size eventually outgrows one instance, shortcodes can be partitioned by a hash of the code itself, since lookups are always by exact key and don't need cross-shard queries. Short-code generation avoids collisions by drawing from pre-allocated ranges or using a distributed ID scheme, rather than relying on a single counter on one machine.

Exercise 2 — Vertically-Scaled Order Service Hitting Its Ceiling

A food-delivery order service has run on a single, vertically-scaled MySQL server for two years. It is now on the largest available instance size, and write latency degrades noticeably during lunch and dinner peaks. Redesign it.

Identify: requirements, traffic pattern, storage, architecture, bottlenecks, scaling strategy, failure scenarios, and trade-offs — before reading further.

Expected Thinking Process

- Recognize the traffic pattern is predictable and spiky (lunch/dinner), not constantly growing — which affects whether auto-scaling or pre-provisioning is more appropriate.
- Separate read load (checking order status, restaurant menus) from write load (placing orders, updating order state) — they need different fixes.
- Note that "biggest instance size" reached means vertical scaling is exhausted for this database — the next lever has to be horizontal, at the data tier.
- Consider whether all writes need to hit the database synchronously, or whether some can be smoothed with a queue.
- Think about what a good shard key would be (e.g. restaurant ID or region) given that most queries are naturally scoped that way.

Reference Solution

Move read-heavy traffic (menus, order status polling) off the primary database entirely: add a cache for menu data (which changes rarely) and add read replicas for order-status reads. For the write path, introduce a message queue between order placement and order processing, so a burst of incoming orders is absorbed and processed at a sustainable rate rather than hitting the database as a synchronous spike. For the underlying data store, shard orders by region or restaurant ID, since almost every query (a restaurant's orders, a region's active orders) is naturally scoped to one shard, keeping cross-shard queries rare. This combination — cache and replicas for reads, a queue to smooth write bursts, and sharding by a naturally-aligned key — addresses the actual bottleneck instead of buying yet another vertical upgrade that will hit the same ceiling again next quarter.

12. Quick Interview Revision Sheet

30-Second Revision

Scalability = handling more work by adding resources. Horizontal = more machines, near-limitless, needs statelessness + a load balancer, no single point of failure. Vertical = a bigger machine, simple, hard ceiling, usually needs downtime, single point of failure.

2-Minute Revision

- Two directions: horizontal (scaling out — add nodes) and vertical (scaling up — bigger node).
- Horizontal scaling requires a stateless app tier, a load balancer, and eventually a scalable data tier (cache, replicas, or sharding).
- Vertical scaling needs zero redesign but has a hard ceiling, usually needs downtime to resize, and is a single point of failure.
- Databases are harder to scale horizontally than stateless app servers — Cassandra/MongoDB are built for it; MySQL/PostgreSQL need extra tooling (replicas, then sharding).
- Horizontal scaling gives diminishing returns once a shared bottleneck (a single DB, a lock) remains — this is Amdahl's Law in practice.
- A good answer always names where state lives and what the next bottleneck will be — not just "add more servers."
- Real systems commonly scale vertically first, then move to horizontal once the ceiling, availability need, or cost curve forces it.

Interview Cheat Sheet

	Horizontal	Vertical
Change	# of machines	Size of one machine
Ceiling	Practically far away	Biggest instance available
Downtime to add capacity	No	Usually yes
Single point of failure	No (if redundant)	Yes
Prerequisite	Statelessness / partitioning	None
Complexity / ops overhead	Higher	Lower

13. Senior Engineer Perspective

How a Senior Engineer Should Think About This

An engineer with 7–12 years of experience doesn't start from "horizontal is modern, vertical is legacy." They start from requirements: current traffic, growth trajectory, availability SLA, team size, and budget. Vertical scaling is often the right first move precisely because it costs nothing in engineering time — a senior engineer isn't afraid to recommend the "boring" option when the data doesn't justify the complex one yet.

What changes with seniority is the ability to see the second-order effects. Choosing horizontal scaling isn't just "add more servers" — it's accepting an ongoing tax: service discovery, deployment orchestration, distributed tracing and centralized logging (because you can no longer SSH into "the" server to read a log file), consistent configuration across a fleet, and a security model where every instance-to-instance call needs its own authentication rather than trusting "localhost." A senior engineer weighs that tax against the alternative — staying vertical and accepting a lower ceiling and an SLA-limiting single point of failure — rather than assuming one option is free.

Failure modes get equal weight to the happy path. For a horizontally-scaled system: what happens when a node fails mid-request, when the load balancer's health checks lag reality, when auto-scaling reacts too slowly to a spike, or when a "stateless" service turns out to have a subtle sticky dependency someone forgot about. For a vertically-scaled system: what the actual recovery time objective is when that one machine goes down, and whether the business has agreed to that number.

Cost is treated as an engineering input, not an afterthought — the cost curve of ever-bigger instances versus the cost of the engineering time to make a system horizontally scalable is a real trade-off, and a senior engineer can usually estimate both well enough to make the conversation concrete instead of

abstract. Observability is treated as a prerequisite for scaling out, not a nice-to-have added later — you cannot safely run twenty instances if you can't see, in one place, what all twenty are doing. And maintainability is judged by whether the team can operate the result at 3 a.m. without the one person who built it — which is often the deciding factor between an elegant horizontal design on paper and one that actually survives production.

The senior-level default

Never present this as "always scale horizontally" or "vertical scaling is outdated." The correct senior answer is always: it depends on the requirements — and then show the reasoning: Requirement → Decision → Reason → Trade-off → Alternative.